

Backend B2C

Technical Requirements

Document

Architecture · API Design · Security · Scalability · DevOps

DOCUMENT TYPE	Technical Requirements Specification
PRODUCT	Tabi Platform — B2C Backend
VERSION	1.0.0
STATUS	Draft — Internal Review
STACK	FastAPI · Python 3.11+ · PostgreSQL · GCP · Kubernetes
DATE	June 04, 2026
CLASSIFICATION	CONFIDENTIAL

Table of Contents

01. Executive Summary

- 1.1 Product Vision
- 1.2 Document Scope
- 1.3 Strategic Objectives
- 1.4 Technology Summary

02. Backend Context & System Boundaries

- 2.1 System Context Diagram
- 2.2 Integration Boundaries
- 2.3 Deployment Environments

03. Technical Objectives

- 3.1 Performance Objectives
- 3.2 Quality Objectives
- 3.3 Architectural Objectives

04. General Architecture

- 4.1 Architecture Style
- 4.2 Directory Structure
- 4.3 Layered Architecture

05. Domain Definition & Module Design

- 5.1 Core Domains
- 5.2 Domain Interaction Rules
- 5.3 Reservation State Machine

06. REST API Design

- 6.1 Design Principles
- 6.2 Authentication Endpoints
- 6.3 Restaurant Endpoints
- 6.4 Reservation Endpoints
- 6.5 Error Schema

07. Data Design & Entity Model

- 7.1 Core Entities (User, Restaurant, Reservation)
- 7.2 Indexing Strategy

08. Functional Requirements

- 8.1 Auth FR-001–FR-006
- 8.2 Discovery FR-010–FR-015
- 8.3 Reservations FR-020–FR-027

- 8.4 User Profile FR-030–FR-034

09. Non-Functional Requirements

- 9.1 Performance NFR-001–NFR-005
- 9.2 Reliability NFR-010–NFR-014
- 9.3 Security NFR-020–NFR-025
- 9.4 Scalability NFR-030–NFR-034

10. Security Architecture

- 10.1 Auth Flow
- 10.2 JWT Structure
- 10.3 Security Controls Matrix
- 10.4 Data Privacy

11. Observability & Logging

- 11.1 Three Pillars Strategy
- 11.2 Log Format
- 11.3 Alerting Rules

12. Kubernetes & Cloud Infrastructure

- 12.1 GKE Cluster Architecture
- 12.2 Deployment Manifest
- 12.3 HPA Config
- 12.4 GCP Managed Services

13. DevOps & Testing Strategy

- 13.1 CI/CD Pipeline
- 13.2 Testing Pyramid
- 13.3 SLAs & SLOs

14. Technical Risks & Roadmap

- 14.1 Risk Register
- 14.2 Technical Roadmap
- 14.3 Microservices Migration

15. Async Services & External Integrations

- 15.1 Celery Tasks
- 15.2 Payment Integration Flow
- 15.3 Idempotency Strategy

SECTION 01

Executive Summary

This document constitutes the official Backend Technical Requirements Specification (TRS) for the **Tabi Platform B2C Backend**, the server-side infrastructure layer responsible for powering all consumer-facing interactions of the Tabi restaurant reservation ecosystem. It is intended for engineering teams, technical leadership, DevOps engineers, security auditors, and external technology partners involved in the design, implementation, and operation of the system.

1.1 Product Vision

Tabi is a next-generation restaurant reservation platform engineered to become the dominant booking infrastructure in Medellín and Colombia. The platform enables consumers to discover, evaluate, and confirm restaurant reservations in under 60 seconds, delivering a user experience comparable to global leaders such as OpenTable and Resy while being purpose-built for the Colombian market and its specific logistical, cultural, and technical requirements.

1.2 Document Scope

This specification covers exclusively the **B2C backend layer** — the set of APIs, business logic services, data models, security controls, and infrastructure components that serve end-user clients (mobile apps and web). B2B restaurant management APIs, CMS, and analytics dashboards are governed by separate specifications.

1.3 Strategic Objectives

Objective	Description	Success Metric
Market Leadership	Become #1 restaurant reservation platform in Medellín within 18 months	50,000+ active users, 500+ restaurants
Speed	Full reservation flow completed in < 60 seconds end-to-end	P95 booking completion time ≤ 58s
Reliability	99.9% uptime SLA for core reservation endpoints	< 43.8 min downtime/month
Scalability	Support 10x traffic spikes without degradation	Auto-scale from 100 to 1,000 RPS
Security	Zero critical vulnerabilities in production	Quarterly pen-test clean pass

1.4 Technology Summary

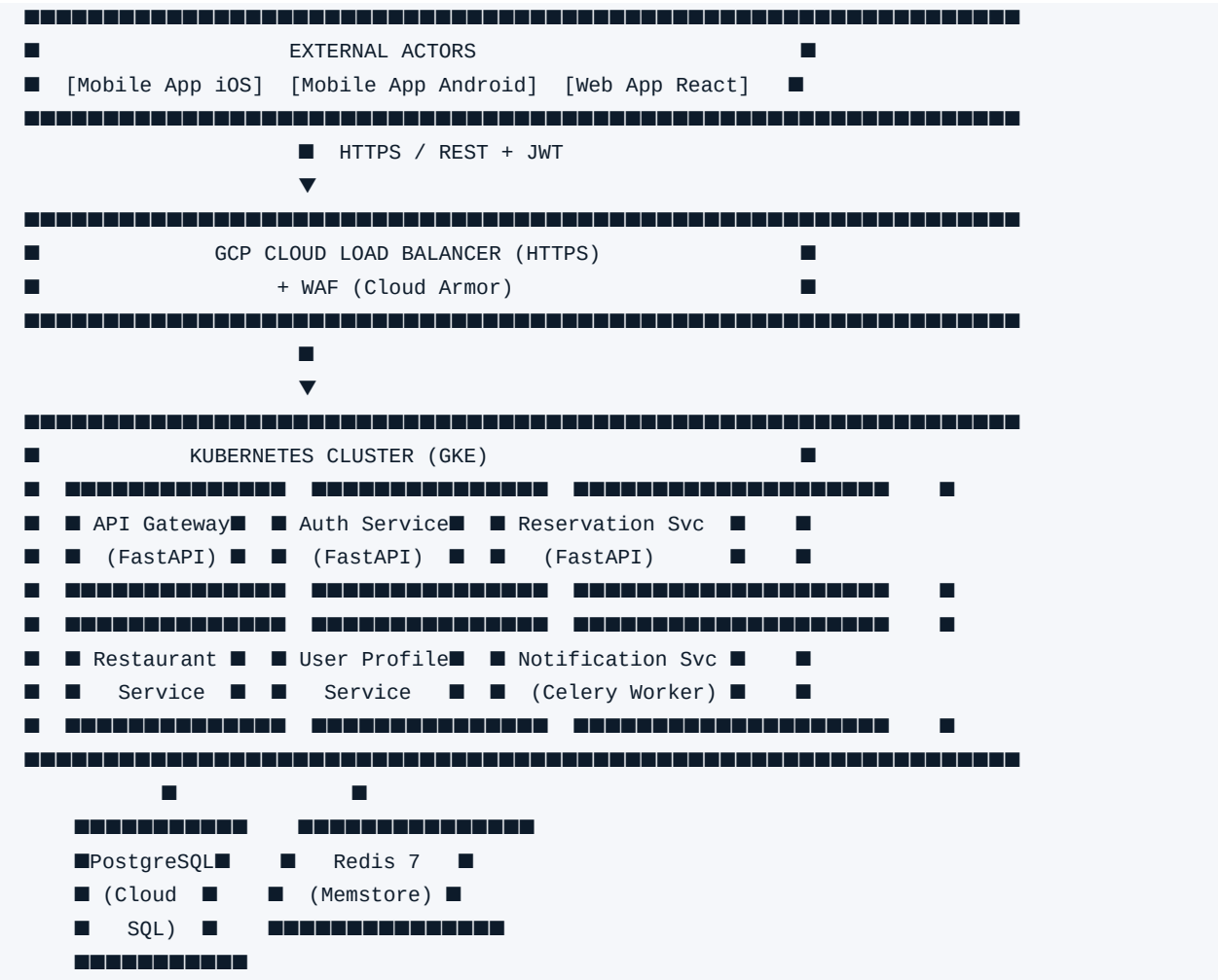
Layer	Technology	Rationale
API Framework	FastAPI 0.110+	Async-first, OpenAPI native, high throughput
Language	Python 3.11+	Performance improvements, modern typing
Database	PostgreSQL 15+	ACID compliance, JSON support, mature ecosystem
ORM	SQLAlchemy 2.0 (async)	Type-safe queries, connection pooling
Cache	Redis 7+	Session store, rate limiting, pub/sub
Container	Docker + Kubernetes (GKE)	Portability, horizontal scaling
Cloud	Google Cloud Platform	Regional presence, managed services
Auth	JWT (RS256) + OAuth 2.0	Stateless auth, third-party SSO
Async	Celery + Redis	Background jobs, notifications
Observability	OpenTelemetry + GCP Monitoring	Distributed tracing, alerting

SECTION 02

Backend Context & System Boundaries

2.1 System Context Diagram

The following describes the top-level system context for the Tabi B2C backend:



2.2 Integration Boundaries

External System	Integration Type	Direction	Purpose
Firebase Auth / Google OAuth	OAuth 2.0	Inbound	Social login SSO
Twilio / AWS SNS	REST API	Outbound	SMS confirmation & OTP
SendGrid	REST API	Outbound	Transactional email

External System	Integration Type	Direction	Purpose
Google Maps Platform	REST API	Outbound	Geocoding, place details
Wompi / PSE / Stripe	REST API	Bidirectional	Payment processing
Firebase Cloud Messaging	REST API	Outbound	Push notifications
Sentry	SDK	Outbound	Error tracking
GCP Pub/Sub	Managed	Internal	Async event bus

2.3 Deployment Environments

Environment	Purpose	GCP Region	Domain
Development (DEV)	Feature development & unit testing	us-central1	api-dev.tabi.co
Staging (STG)	Integration & QA testing	us-central1	api-staging.tabi.co
Production (PRD)	Live user traffic	us-east1 + southamerica-east1	api.tabi.co

SECTION 03

Technical Objectives

3.1 Performance Objectives

ID	Objective	Target	Priority
TO-01	API response time for search queries	P95 < 300ms	Critical
TO-02	Reservation creation latency	P95 < 500ms	Critical
TO-03	Authentication endpoint throughput	≥ 500 req/s per pod	High
TO-04	Database query execution	P99 < 100ms for indexed queries	High
TO-05	Cache hit ratio for restaurant listings	≥ 85%	Medium
TO-06	Background job queue lag	< 5 seconds under normal load	Medium

3.2 Quality Objectives

ID	Objective	Target	Measurement
TO-07	Unit test coverage	≥ 85%	pytest-cov report
TO-08	Integration test coverage for critical paths	100%	CI gate
TO-09	API documentation completeness	100% endpoints documented	OpenAPI spec
TO-10	Linting & type safety	Zero mypy errors on strict mode	CI gate
TO-11	Dependency vulnerability score	Zero critical CVEs in production	Snyk scan
TO-12	Mean Time to Recovery (MTTR)	< 30 minutes	Incident log

3.3 Architectural Objectives

- Maintain strict separation between domain logic and infrastructure concerns (Clean Architecture).
- All modules must be independently testable and deployable without cross-module database joins.
- Zero downtime deployments via rolling updates in Kubernetes.

- • All secrets managed via GCP Secret Manager — no hardcoded credentials.
- • API-first design: every feature exposed through versioned REST endpoints before UI implementation.
- • Observability by default: all services emit structured logs, metrics, and distributed traces.

SECTION 04

General Architecture

4.1 Architecture Style

The Tabi B2C backend adopts a **Modular Monolith architecture** as the initial deployment strategy, designed from day one with clear domain boundaries that facilitate future migration to independent microservices without a full rewrite. Each domain module is self-contained with its own models, services, routers, and DTOs, communicating with other modules exclusively through defined service interfaces, never through direct database cross-joins.

4.2 Application Directory Structure

```

tabi-backend/
├── app/
│   ├── main.py                # FastAPI application factory
│   ├── config/
│   │   ├── settings.py        # Pydantic BaseSettings
│   │   ├── database.py        # SQLAlchemy async engine
│   │   ├── redis.py           # Redis connection pool
│   │   └── models/            # SQLAlchemy ORM models
│   │       ├── user.py
│   │       ├── restaurant.py
│   │       ├── reservation.py
│   │       └── base.py        # Base declarative model
│   ├── dtos/                  # Pydantic request/response schemas
│   │   ├── user_dto.py
│   │   ├── restaurant_dto.py
│   │   └── reservation_dto.py
│   ├── controllers/           # Request handlers (thin layer)
│   ├── routers/               # FastAPI APIRouter definitions
│   │   └── v1/
│   │       ├── auth.py
│   │       ├── users.py
│   │       ├── restaurants.py
│   │       └── reservations.py
│   ├── services/              # Business logic orchestration
│   │   ├── auth_service.py
│   │   ├── reservation_service.py
│   │   └── restaurant_service.py
│   ├── implements/           # Repository pattern (DB access)
│   │   ├── user_repository.py
│   │   ├── reservation_repository.py
│   │   └── restaurant_repository.py
│   └── infrastructure/        # External integrations

```

```

■ ■ ■ ■ email/
■ ■ ■ ■ sms/
■ ■ ■ ■ payments/
■ ■ ■ ■ notifications/
■ ■ ■ ■ tests/
■ ■ ■ ■ unit/
■ ■ ■ ■ integration/
■ ■ ■ ■ e2e/
■ ■ ■ ■ alembic/ # DB migrations
■ ■ ■ ■ docker/
■ ■ ■ ■ Dockerfile
■ ■ ■ ■ docker-compose.yml
■ ■ ■ ■ k8s/ # Kubernetes manifests
■ ■ ■ ■ .github/workflows/ # CI/CD pipelines
■ ■ ■ ■ pyproject.toml

```

4.3 Layered Architecture

PRESENTATION LAYER		
Routers + Controllers + DTOs (Pydantic)		
Input validation · Response shaping		
APPLICATION LAYER		
Services · Use Cases · Orchestration		
Business rules · Transaction coordination		
DOMAIN LAYER		
Entities · Value Objects · Domain Events		
Pure Python – no framework deps		
INFRASTRUCTURE LAYER		
Repositories · ORM · Redis · External APIs		
Database adapters · Message brokers		

SECTION 05

Domain Definition & Module Design

5.1 Core Domains

Domain	Responsibility	Key Entities	External Deps
Identity & Auth	User registration, login, token lifecycle, OAuth SSO	User, Session, OAuthProvider	Google OAuth, Firebase
User Profile	Profile CRUD, preferences, dietary restrictions, history	UserProfile, Preference, Address	Google Maps
Restaurant Catalog	Restaurant discovery, search, filtering, details	Restaurant, Menu, Schedule, Table	Google Maps, GCS
Reservation	Booking creation, modification, cancellation, slots	Reservation, TimeSlot, WaitList	Notification Svc
Notification	Push, SMS, email delivery and template management	Notification, Template, Channel	FCM, Twilio, SendGrid
Payment	Payment initiation, verification, refunds	Payment, Invoice, Refund	Wompi, Stripe
Review & Rating	Post-visit reviews, ratings aggregation	Review, Rating, MediaAsset	GCS
Search & Discovery	Full-text search, geo-search, recommendations	SearchIndex, GeoPoint	PostgreSQL FTS, Maps

5.2 Domain Interaction Rules

- Domains communicate exclusively through **service interfaces**, never via direct ORM model imports.
- Cross-domain data transfer uses **DTOs** — never raw SQLAlchemy model objects.
- Asynchronous cross-domain events are published to **GCP Pub/Sub** topics.
- The Reservation domain is the **system of record** — it owns all booking state transitions.
- Payment domain events trigger downstream actions in Notification and Reservation domains.

5.3 Reservation State Machine





Transition	Trigger	Actor	Side Effects
PENDING → CONFIRMED	Payment captured successfully	Payment service webhook	Notify user + restaurant via push/SMS/email
PENDING → EXPIRED	Payment timeout (15 min)	Celery scheduled task	Release slot, notify user, log event
CONFIRMED → CANCELLED	User or restaurant cancels	API call	Initiate refund if applicable, notify counterparty
CONFIRMED → NO_SHOW	No check-in within 30min window	Celery scheduled task	Flag user record, notify restaurant
NO_SHOW → COMPLETED	Restaurant marks visit complete	B2B API	Unlock review capability, close reservation

SECTION 06

REST API Design

6.1 API Design Principles

- **Versioning:** All endpoints prefixed with `/api/v1/`. Breaking changes require a new version prefix.
- **Naming:** Plural nouns for collections (`/restaurants`, `/reservations`). Verbs only for RPC-style actions.
- **HTTP Semantics:** Strict adherence to HTTP method semantics — GET is idempotent, POST creates, PUT replaces, PATCH partially updates.
- **Status Codes:** Precise HTTP status codes. 422 for validation errors, 409 for conflicts, 429 for rate limiting.
- **Pagination:** All list endpoints use cursor-based pagination (after, limit parameters).
- **Error Format:** Unified RFC 7807 Problem Details JSON for all errors.

6.2 Authentication Endpoints

Method	Endpoint	Description	Auth Required
POST	<code>/api/v1/auth/register</code>	Register new user with email + password	No
POST	<code>/api/v1/auth/login</code>	Authenticate and receive JWT pair	No
POST	<code>/api/v1/auth/refresh</code>	Refresh access token using refresh token	Refresh Token
POST	<code>/api/v1/auth/logout</code>	Revoke refresh token	Access Token
POST	<code>/api/v1/auth/oauth/google</code>	Initiate Google OAuth 2.0 flow	No
POST	<code>/api/v1/auth/oauth/callback</code>	Handle OAuth provider callback	No
POST	<code>/api/v1/auth/password/reset</code>	Request password reset email	No
PUT	<code>/api/v1/auth/password/reset/{token}</code>	Confirm password reset with token	No
POST	<code>/api/v1/auth/otp/send</code>	Send OTP to phone number	No
POST	<code>/api/v1/auth/otp/verify</code>	Verify OTP and return tokens	No

6.3 Restaurant Endpoints

Method	Endpoint	Description	Auth Required
GET	/api/v1/restaurants	List/search restaurants (geo, filters, FTS)	No
GET	/api/v1/restaurants/{id}	Get restaurant detail with menu + schedule	No
GET	/api/v1/restaurants/{id}/availability	Get available time slots for a date	No
GET	/api/v1/restaurants/{id}/reviews	Paginated list of verified reviews	No
GET	/api/v1/restaurants/featured	Curated featured restaurants list	No
GET	/api/v1/restaurants/nearby	Geo-sorted restaurants by lat/lng	No
GET	/api/v1/categories	List cuisine categories	No
GET	/api/v1/restaurants/search/suggestions	Autocomplete search suggestions	No

6.4 Reservation Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/v1/reservations	Create new reservation (initiates payment)	Access Token
GET	/api/v1/reservations	List authenticated user's reservations	Access Token
GET	/api/v1/reservations/{id}	Get reservation detail	Access Token
PATCH	/api/v1/reservations/{id}	Modify reservation (date/time/party size)	Access Token
DELETE	/api/v1/reservations/{id}	Cancel reservation	Access Token
GET	/api/v1/reservations/{id}/status	Real-time status check (polling)	Access Token
POST	/api/v1/reservations/{id}/check-in	User self check-in QR code verification	Access Token

6.5 Standard Error Response Schema

```
{
  "type": "https://tabi.co/errors/reservation-conflict",
  "title": "Reservation Conflict",
  "status": 409,
  "detail": "The requested time slot is no longer available.",
  "instance": "/api/v1/reservations",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
}
```

```
"timestamp": "2025-06-04T10:30:00Z",  
"errors": []  
}
```

SECTION 07

Data Design & Entity Model

7.1 Core Entity Definitions

User

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Primary identifier (gen_random_uuid())
email	VARCHAR(255)	UNIQUE, NOT NULL	Verified user email address
phone	VARCHAR(20)	UNIQUE, NULLABLE	E.164 format phone number
password_hash	VARCHAR(255)	NULLABLE	Argon2id hash — null for OAuth users
first_name	VARCHAR(100)	NOT NULL	User first name
last_name	VARCHAR(100)	NOT NULL	User last name
avatar_url	TEXT	NULLABLE	GCS public URL
is_verified	BOOLEAN	DEFAULT FALSE	Email/phone verification flag
is_active	BOOLEAN	DEFAULT TRUE	Soft delete / account status
oauth_provider	VARCHAR(50)	NULLABLE	google facebook apple
oauth_sub	VARCHAR(255)	NULLABLE	Provider subject identifier
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Creation timestamp
updated_at	TIMESTAMPTZ	NOT NULL	Last update timestamp

Restaurant

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Primary identifier
name	VARCHAR(200)	NOT NULL	Restaurant commercial name
slug	VARCHAR(200)	UNIQUE, NOT NULL	URL-friendly identifier
description	TEXT	NULLABLE	Full description (max 2000 chars)
category_id	UUID	FK → categories, NOT NULL	Cuisine category

Column	Type	Constraints	Description
address	TEXT	NOT NULL	Full street address
latitude	DECIMAL(10,8)	NOT NULL	WGS84 latitude
longitude	DECIMAL(11,8)	NOT NULL	WGS84 longitude
phone	VARCHAR(20)	NOT NULL	Contact phone
email	VARCHAR(255)	NULLABLE	Contact email
website	TEXT	NULLABLE	Restaurant website URL
price_range	SMALLINT	NOT NULL (1-4)	\$ to \$\$\$\$ price indicator
avg_rating	DECIMAL(3,2)	DEFAULT 0.00	Denormalized average rating
total_reviews	INTEGER	DEFAULT 0	Denormalized review count
is_active	BOOLEAN	DEFAULT TRUE	Visibility flag
is_featured	BOOLEAN	DEFAULT FALSE	Editorial featuring flag
geo_point	GEOGRAPHY(Point, 4326)	NOT NULL	PostGIS spatial index
created_at	TIMESTAMPTZ	NOT NULL	Creation timestamp

Reservation

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Primary identifier
user_id	UUID	FK → users, NOT NULL	Booking user
restaurant_id	UUID	FK → restaurants, NOT NULL	Target restaurant
table_id	UUID	FK → tables, NULLABLE	Assigned table (post-confirm)
status	reservation_status	ENUM, NOT NULL	State machine status
party_size	SMALLINT	NOT NULL (1-20)	Number of guests
reserved_date	DATE	NOT NULL	Reservation date
reserved_time	TIME	NOT NULL	Reservation start time
confirmation_code	VARCHAR(12)	UNIQUE, NOT NULL	Human-readable code
special_requests	TEXT	NULLABLE	Dietary/occasion notes
payment_id	UUID	FK → payments, NULLABLE	Associated payment

Column	Type	Constraints	Description
cancelled_at	TIMESTAMPTZ	NULLABLE	Cancellation timestamp
cancellation_reason	TEXT	NULLABLE	Cancellation reason text
expires_at	TIMESTAMPTZ	NOT NULL	PENDING state expiry (15 min)
created_at	TIMESTAMPTZ	NOT NULL	Creation timestamp
updated_at	TIMESTAMPTZ	NOT NULL	Last update timestamp

7.2 Database Indexing Strategy

Table	Index	Type	Reason
users	idx_users_email	B-Tree UNIQUE	Login lookup
users	idx_users_phone	B-Tree UNIQUE	Phone auth lookup
restaurants	idx_restaurants_geo	GIST (geo_point)	Geo-proximity queries
restaurants	idx_restaurants_slug	B-Tree UNIQUE	Slug-based routing
restaurants	idx_restaurants_category	B-Tree	Category filter
restaurants	idx_restaurants_fts	GIN (tsvector)	Full-text search
reservations	idx_res_user_date	B-Tree (user_id, reserved_date)	User history queries
reservations	idx_res_restaurant_datetime	B-Tree (restaurant_id, reserved_date, reserved_time)	Slot conflict check
reservations	idx_res_status	B-Tree (status, expires_at)	Celery expiry scans
reservations	idx_res_confirmation_code	B-Tree UNIQUE	QR code lookup

SECTION 08

Functional Requirements

8.1 Authentication & Authorization

ID	Requirement	Priority	Acceptance Criteria
FR-001	System shall allow user registration via email/password with email verification	Critical	Verification email sent within 30s; account inactive until verified
FR-002	System shall support Google OAuth 2.0 SSO login	Critical	OAuth flow completes in < 3 steps; JWT issued on success
FR-003	System shall issue RS256-signed JWT access tokens with 15-min expiry	Critical	Token rejected after 900s; RS256 signature verified on each request
FR-004	System shall issue rotating refresh tokens with 30-day expiry	Critical	Refresh token single-use; new pair issued on each refresh call
FR-005	System shall support OTP-based phone login via SMS	High	OTP expires in 5 min; max 3 attempts before lockout
FR-006	System shall enforce rate limiting on auth endpoints	Critical	Max 5 failed attempts per IP per 15 min; 429 returned thereafter

8.2 Restaurant Discovery

ID	Requirement	Priority	Acceptance Criteria
FR-010	System shall support geo-proximity search returning restaurants within a configurable radius	Critical	Returns sorted results by distance; supports up to 50km radius
FR-011	System shall support full-text search across name, description, and cuisine tags	Critical	Results returned in < 300ms; ranked by relevance + rating
FR-012	System shall support multi-dimensional filtering (price range, cuisine, rating, open now)	High	Filters are composable; each filter reduces result set correctly
FR-013	System shall return paginated results with cursor-based pagination	High	Default page size 20; max 100; stable cursor across requests
FR-014	System shall expose real-time availability indicator in listing view	High	Slot availability checked at query time; cached for max 60s
FR-015	System shall support featured restaurant editorial curation	Medium	Featured flag set via B2B admin API; displayed in dedicated endpoint

8.3 Reservation Management

ID	Requirement	Priority	Acceptance Criteria
FR-020	System shall create reservations with atomic slot locking (no double-booking)	Critical	Pessimistic DB lock held during slot check + insert; 409 on conflict
FR-021	System shall automatically expire PENDING reservations after 15 minutes	Critical	Celery task scans every 60s; releases slot and notifies user
FR-022	System shall send multi-channel confirmation (push + SMS + email) on CONFIRMED status	Critical	All three channels attempted; at least one must succeed
FR-023	System shall support reservation modification up to 2 hours before reserved time	High	Original slot released; new slot locked atomically
FR-024	System shall support self-service cancellation with configurable cancellation policy	High	Policy evaluated per restaurant; refund trigger sent if applicable
FR-025	System shall generate unique QR code per confirmed reservation for check-in	High	QR encodes signed JWT; verifiable offline; expires at reservation end
FR-026	System shall support waitlist enrollment when restaurant is fully booked	Medium	User notified immediately if slot opens; 10-min acceptance window
FR-027	System shall send reminder notifications 24h and 2h before reservation	High	Celery scheduled; respects user notification preferences

8.4 User Profile & Preferences

ID	Requirement	Priority	Acceptance Criteria
FR-030	System shall allow users to manage dietary restrictions and preferences	High	Preferences stored per user; surfaced in restaurant recommendations
FR-031	System shall maintain a complete history of past and upcoming reservations	High	Paginated history; filterable by status and date range
FR-032	System shall allow users to save favorite restaurants	Medium	Favorite list persisted; accessible via dedicated endpoint
FR-033	System shall allow users to submit post-visit reviews with rating	High	Review only unlockable after COMPLETED reservation; one per visit
FR-034	System shall allow users to manage notification channel preferences	Medium	Per-channel toggle (push/SMS/email) stored; respected by notification service

SECTION 09

Non-Functional Requirements

9.1 Performance

ID	Requirement	Target	Test Method
NFR-001	Restaurant search API response time	P95 < 300ms at 500 concurrent users	k6 load test
NFR-002	Reservation creation response time	P95 < 500ms at 200 concurrent users	k6 load test
NFR-003	Authentication endpoint throughput	≥ 1,000 req/s across 3 replicas	Locust benchmark
NFR-004	Database connection pool saturation	< 70% utilization under peak load	CloudSQL metrics
NFR-005	Redis cache response time	P99 < 5ms for GET operations	Memstore metrics

9.2 Reliability & Availability

ID	Requirement	Target	Recovery Mechanism
NFR-010	Core API uptime SLA	99.9% monthly (< 43.8 min downtime)	Multi-zone GKE + health checks
NFR-011	Database durability	Zero data loss (RPO = 0s for transactions)	CloudSQL synchronous replication
NFR-012	Graceful degradation on cache miss	API returns stale data, never fails	Cache-aside with DB fallback
NFR-013	Circuit breaker for external services	Open after 5 failures in 30s window	Tenacity library
NFR-014	Automatic pod restart on health check failure	Pod replaced within 30 seconds	Kubernetes liveness probe

9.3 Security

ID	Requirement	Standard	Verification
NFR-020	All data in transit encrypted	TLS 1.3 minimum	SSL Labs A+ rating

ID	Requirement	Standard	Verification
NFR-021	Sensitive data at rest encrypted	AES-256 (CloudSQL encryption)	GCP encryption policy
NFR-022	No secrets in code or Docker images	GCP Secret Manager + Workload Identity	Trufflehog scan in CI
NFR-023	SQL injection prevention	SQLAlchemy parameterized queries only	SAST + manual review
NFR-024	OWASP Top 10 compliance	Full Top 10 mitigation checklist	Quarterly pen test
NFR-025	PII data handling compliance	Colombia Ley 1581 (Habeas Data)	Legal + DPO review

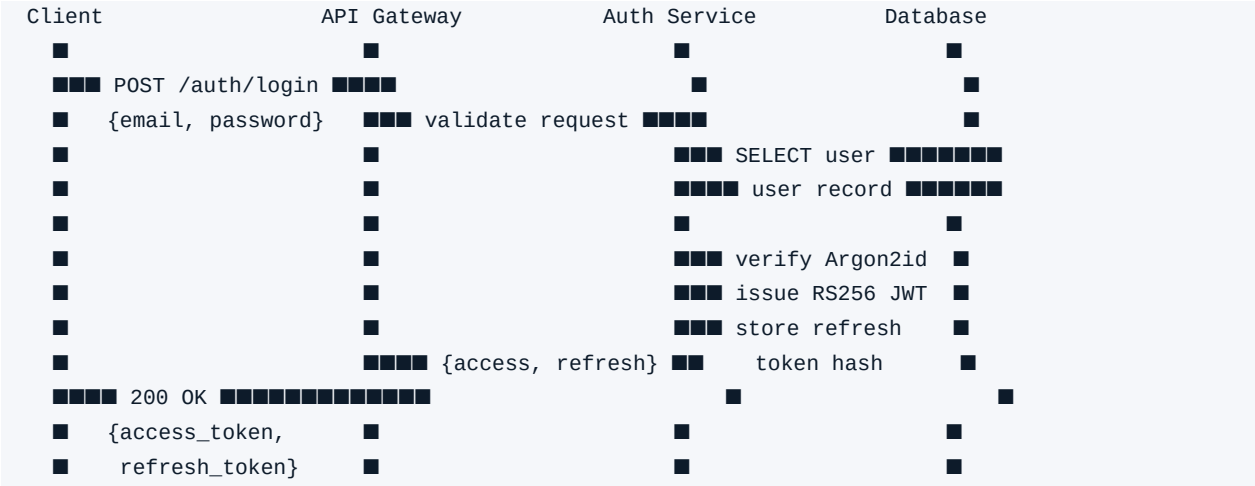
9.4 Scalability

ID	Requirement	Target	Implementation
NFR-030	Horizontal pod autoscaling	Scale from 2 to 20 replicas based on CPU/RPS	HPA in Kubernetes
NFR-031	Stateless API design	Zero session state in API pods	All state in Redis/DB
NFR-032	Database read scaling	Read replicas for heavy read queries	CloudSQL read replicas
NFR-033	Background job scalability	Celery workers scale independently from API	Separate Deployment in K8s
NFR-034	Multi-region readiness	Architecture prepared for multi-region by Q3 2026	GCP regional design

SECTION 10

Security Architecture

10.1 Authentication Flow



10.2 JWT Token Structure

```
// Access Token Payload (RS256, 15-minute expiry)
{
  "sub": "user-uuid-here",
  "iat": 1717491600,
  "exp": 1717492500,
  "jti": "unique-token-id",
  "roles": ["user"],
  "email_verified": true,
  "iss": "https://api.tabi.co"
}
```

10.3 Security Controls Matrix

Threat	Control	Implementation	Severity Mitigated
Credential stuffing	Rate limiting + CAPTCHA on auth	SlowAPI + reCAPTCHA v3	Critical
JWT token theft	Short expiry + refresh rotation	15-min access / 30-day refresh	Critical

Threat	Control	Implementation	Severity Mitigated
SQL injection	Parameterized queries only	SQLAlchemy ORM enforced	Critical
XSS via API responses	Content-Type enforcement + input sanitization	Pydantic validators + bleach	High
CSRF	SameSite cookie policy + CSRF token for web	FastAPI middleware	High
Insecure Direct Object Reference	Resource ownership validation in every service layer	Custom dependency injection check	Critical
Secrets exposure	GCP Secret Manager + Workload Identity	No env vars with secrets	Critical
DDoS	Cloud Armor WAF + rate limiting + CDN	GCP Cloud Armor rules	High
Man-in-the-Middle	TLS 1.3 enforced; HSTS preload	Cloud Load Balancer config	Critical
Privilege escalation	RBAC with principle of least privilege	FastAPI dependency per role	Critical

10.4 Data Privacy (Ley 1581 Compliance)

- All PII collected under explicit informed consent at registration; consent record stored immutably.
- Users may request full data export (portability) via authenticated endpoint within 24h SLA.
- Users may request account deletion; all PII anonymized within 30 days; anonymized analytics retained.
- Phone numbers and emails stored hashed in analytics pipelines; raw values only in transactional DB.
- Data Processing Agreement (DPA) in place with all third-party processors (Twilio, SendGrid, etc.).

SECTION 11

Observability & Logging

11.1 Observability Strategy (Three Pillars)

Pillar	Tool	Scope	Retention
Metrics	GCP Cloud Monitoring + Prometheus	RED metrics per endpoint (Rate, Errors, Duration)	90 days
Logs	GCP Cloud Logging (structured JSON)	All request/response, errors, audit events	30 days hot / 1 year cold
Traces	OpenTelemetry + GCP Cloud Trace	Distributed traces across all service calls	14 days

11.2 Structured Log Format

```
{
  "timestamp": "2025-06-04T10:30:00.123Z",
  "level": "INFO",
  "service": "reservation-service",
  "pod": "reservation-7d4b9c-xk2p9",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "span_id": "00f067aa0ba902b7",
  "user_id": "usr_abc123",
  "request_id": "req_xyz789",
  "method": "POST",
  "path": "/api/v1/reservations",
  "status_code": 201,
  "duration_ms": 142,
  "message": "Reservation created successfully"
}
```

11.3 Key Alerting Rules

Alert Name	Condition	Severity	Action
High Error Rate	5xx rate > 1% over 5-min window	P1 Critical	PagerDuty page on-call engineer
High Latency	P95 latency > 500ms for 10 min	P2 High	Slack alert #backend-alerts
Pod CrashLoop	Pod restart > 3 times in 10 min	P1 Critical	PagerDuty + auto-scale attempt
DB Connection Pool	Pool usage > 85% for 5 min	P2 High	Slack alert + scale read replica

Alert Name	Condition	Severity	Action
Reservation Expiry Queue Lag	Celery queue depth > 500 for 5 min	P2 High	Scale Celery workers
Failed Payment Webhook	Webhook retry exhausted	P2 High	Slack + manual intervention ticket
Security: Auth Spike	Auth failures > 100/min from single IP	P1 Critical	Cloud Armor auto-block IP

SECTION 12

Kubernetes & Cloud Infrastructure

12.1 GKE Cluster Architecture

```
GKE CLUSTER — us-east1 (Production)
■■■ Node Pool: api-pool (n2-standard-4, min:3, max:10, auto-scale)
■   ■■■ Deployment: tabi-api           (replicas: 3-10)
■   ■■■ Deployment: tabi-auth          (replicas: 2-6)
■   ■■■ Deployment: tabi-notifications (replicas: 2-4)
■■■ Node Pool: worker-pool (n2-standard-2, min:2, max:8)
■   ■■■ Deployment: tabi-celery-workers (replicas: 2-8)
■■■ Services:
■   ■■■ ClusterIP:    internal service mesh communication
■   ■■■ LoadBalancer: GCP HTTPS Load Balancer (external)
■■■ ConfigMaps:    non-sensitive configuration per environment
■■■ Secrets:       Workload Identity → GCP Secret Manager
■■■ Ingress:       GKE Ingress + Cloud Armor WAF
```

12.2 API Deployment Manifest (Condensed)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tabi-api
  namespace: production
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  template:
    spec:
      containers:
        - name: tabi-api
          image: gcr.io/tabi-prod/api:{{IMAGE_TAG}}
          resources:
            requests: {cpu: 250m, memory: 512Mi}
            limits: {cpu: 1000m, memory: 1Gi}
          livenessProbe:
            httpGet: {path: /health/live, port: 8000}
            initialDelaySeconds: 10
            periodSeconds: 10
```

```
readinessProbe:
  httpGet: {path: /health/ready, port: 8000}
  initialDelaySeconds: 5
  periodSeconds: 5
```

12.3 Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: tabi-api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: tabi-api
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: {type: Utilization, averageUtilization: 65}
    - type: Pods
      pods:
        metric: {name: http_requests_per_second}
        target: {type: AverageValue, averageValue: 200}
```

12.4 GCP Managed Services

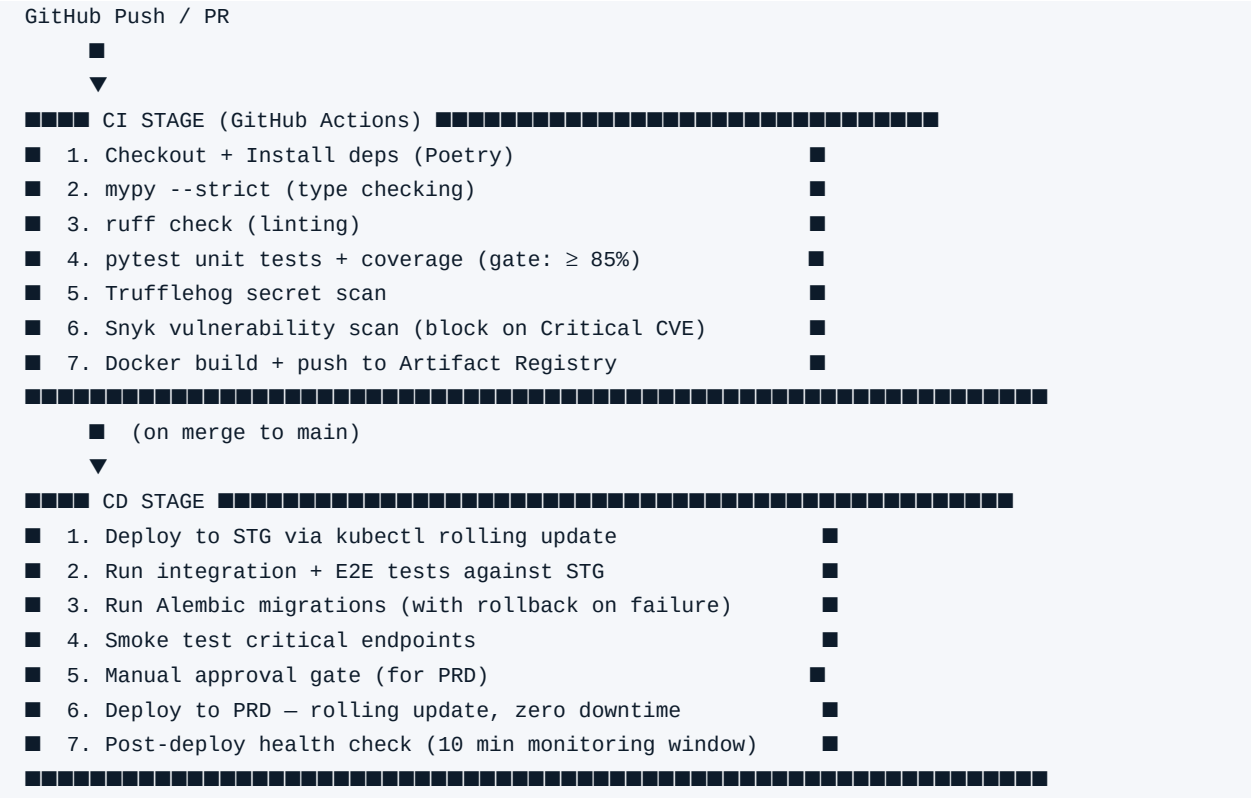
Service	GCP Product	Configuration	Purpose
PostgreSQL	Cloud SQL for PostgreSQL 15	HA, 2 read replicas, automated backups	Primary data store
Redis	Memorystore for Redis 7	Standard tier, 4GB, HA enabled	Cache + session + queue
Container Registry	Artifact Registry	Vulnerability scanning enabled	Docker image storage
Secrets	Secret Manager	Automatic rotation, Workload Identity	All credentials
Object Storage	Cloud Storage	Multi-region bucket, CDN-enabled	Media assets, exports
Load Balancer	Cloud Load Balancing	HTTPS, Cloud Armor WAF attached	Traffic entry point

Service	GCP Product	Configuration	Purpose
Pub/Sub	Cloud Pub/Sub	At-least-once delivery, 7-day retention	Async event bus
Monitoring	Cloud Monitoring + Alerting	Custom dashboards, PagerDuty integration	Observability

SECTION 13

DevOps & Testing Strategy

13.1 CI/CD Pipeline



13.2 Testing Pyramid

Level	Type	Tool	Coverage Target	Run Frequency
Unit	Domain logic, service methods, DTO validation	pytest + pytest-asyncio	≥ 85%	Every push
Integration	DB queries, Redis operations, repository layer	pytest + TestContainers	100% critical paths	Every PR
Contract	API schema validation against OpenAPI spec	schemathesis	All endpoints	Every PR
E2E	Full reservation flow, auth flow, payment flow	pytest + httpx against STG	Core user journeys	Every merge to main

Level	Type	Tool	Coverage Target	Run Frequency
Load	Performance benchmarks at target concurrency	k6 + Locust	All NFR performance targets	Weekly + pre-release
Security	SAST, dependency scan, pen test simulation	Bandit + Snyk + OWASP ZAP	OWASP Top 10	Weekly SAST / Quarterly pen test

13.3 SLAs & SLOs

Service	SLO	SLA	Error Budget (30 days)
Core API (all endpoints)	99.9% availability	99.5% contractual	43.8 min
Reservation creation	99.95% success rate	99.9%	21.9 min
Authentication	99.99% availability	99.9%	4.38 min
Notification delivery	99% delivered within 60s	95%	7.2 hours
Database (CloudSQL)	99.95% availability (GCP SLA)	99.95%	21.9 min

SECTION 14

Technical Risks & Roadmap

14.1 Risk Register

ID	Risk	Probability	Impact	Mitigation
TR-01	Double-booking race condition at high concurrency	Medium	Critical	Pessimistic DB lock (SELECT FOR UPDATE) + idempotency keys
TR-02	Payment webhook delivery failure	Medium	High	Webhook retry with exponential backoff + idempotent handler
TR-03	Cloud SQL failover latency during HA promotion	Low	High	Read replicas + connection retry logic + pgBouncer
TR-04	Third-party SMS/notification provider outage	Medium	Medium	Multi-provider fallback (Twilio → AWS SNS) with circuit breaker
TR-05	Celery worker processing lag during peak hours	Medium	Medium	Separate HPA for workers + priority queue configuration
TR-06	Python GIL limiting CPU-bound concurrency	Low	Medium	FastAPI async I/O handles bottleneck; CPU tasks offloaded to Celery
TR-07	Kubernetes cluster upgrade breaking API compatibility	Low	High	Staging mirrors production K8s version; staged upgrades
TR-08	Schema migration failure in zero-downtime deploy	Low	Critical	Backward-compatible migrations; two-phase deploy strategy

14.2 Technical Roadmap

Phase	Timeline	Deliverable	Success Criteria
Phase 0 — Foundation	Month 1-2	Local dev environment, CI/CD pipeline, DB schema v1, auth endpoints	CI green, auth working, DB migrated
Phase 1 — Core MVP	Month 3-4	Restaurant catalog API, reservation CRUD, basic notifications	End-to-end booking flow functional
Phase 2 — Production Ready	Month 5-6	Payment integration, QR check-in, GKE production deploy, observability	SLOs met under load test
Phase 3 — Growth Features	Month 7-9	Waitlist, reviews, geo-search optimization, recommendation engine	P95 search < 200ms
Phase 4 — Scale	Month 10-12	Multi-region deploy, read replicas, advanced caching, CDN	10x traffic handled without degradation

Phase	Timeline	Deliverable	Success Criteria
Phase 5 — Microservices	Month 13-18	Extract Notification and Payment as independent services	Independent deploys per service

14.3 Microservices Migration Strategy

The modular monolith architecture was designed with explicit seam points to facilitate incremental extraction of domains into independent microservices. The migration follows the **Strangler Fig Pattern**: new traffic is progressively routed to the extracted service while the monolith handles the remainder, until the extracted service handles 100% and the monolith code is deleted.

Service	Extraction Priority	Rationale	Estimated Effort
Notification Service	First (Month 13)	Already isolated; independent scaling needs	3 weeks
Payment Service	Second (Month 14)	Regulatory isolation; separate compliance boundary	4 weeks
Search Service	Third (Month 15)	Elasticsearch migration; dedicated scaling profile	6 weeks
User/Auth Service	Fourth (Month 17)	Shared across all domains; highest complexity	8 weeks
Restaurant Catalog	Fifth (Month 18)	Stable domain; lower urgency	4 weeks

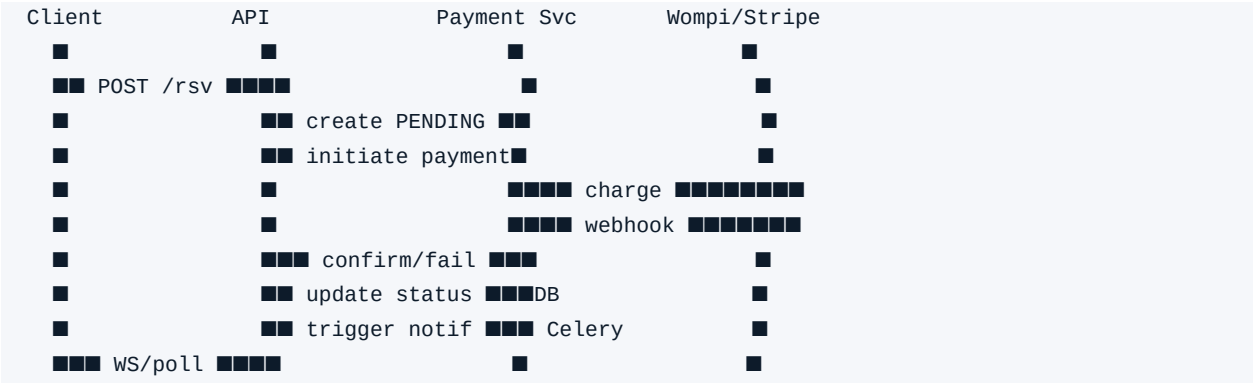
SECTION 15

Async Services & External Integrations

15.1 Celery Task Architecture

Task	Queue	Schedule	Retry Policy
expire_pending_reservations	reservations.expiry	Every 60 seconds	Max 3 retries, no backoff (time-sensitive)
send_push_notification	notifications.push	Immediate on event	Max 5 retries, exponential backoff 2x
send_sms_confirmation	notifications.sms	Immediate on event	Max 3 retries, 30s delay
send_email_confirmation	notifications.email	Immediate on event	Max 5 retries, exponential backoff
send_reservation_reminder_24h	notifications.scheduled	Daily at 08:00 local	Max 3 retries
send_reservation_reminder_2h	notifications.scheduled	Hourly scan	Max 3 retries
update_restaurant_avg_rating	analytics.reviews	Immediate on review submit	Max 3 retries, 60s delay
process_refund	payments.refunds	Immediate on cancellation	Max 5 retries, 5-min delay

15.2 Payment Integration Flow



15.3 Idempotency Strategy

All payment and reservation mutation endpoints require an **Idempotency-Key** header (UUID v4 generated by client). The server stores the key in Redis with a 24-hour TTL. Duplicate requests with the same key return the original response without re-executing the operation. This prevents double charges and duplicate reservations in network retry scenarios.